

Medical Report Summarizer

G. Vinay Kumar Reddy | Reg. No: 192472093

Design of a structured output for hospital medical report summarization, and how the prompt prevents the model from inventing information.

1. Structured Output Design

The output is designed as a strict, fixed JSON schema so it can be parsed reliably by hospital systems such as EHR software and clinician dashboards.

```
{
  "patient_summary": {
    "patient_id": "string or null",
    "age": "integer or null",
    "gender": "string or null",
    "reason_for_visit": "string or null",
    "relevant_history": "string or null"
  },
  "findings": [
    {
      "test_or_exam": "string",
      "result": "string",
      "reference_range": "string or null",
      "abnormal_flag": "true | false | unknown"
    }
  ],
  "observations": [
    "string (clinician's noted observations)"
  ],
  "recommended_follow_up": [
    {
      "action": "string",
      "timeframe": "string or null",
      "reason": "string or null"
    }
  ],
  "information_not_found": [
    "string (fields the model could not locate in the source report)"
  ]
}
```

Why each section is separated

Section	Purpose
patient_summary	Demographic and visit context, kept minimal to avoid clinical interpretation.
findings	Atomic, test-by-test facts (labs, imaging, vitals) kept separate so nothing is blended or reinterpreted.
observations	Qualitative clinician notes, separated from hard findings so inference is not mixed with data.

Section	Purpose
recommended_follow_up	Only actions explicitly stated in the source report (referrals, repeat tests, medication changes).
information_not_found	An explicit gap list; the core anti-hallucination mechanism (see Section 2).

2. Preventing the Model From Inventing Information

a) Explicit grounding instruction

The prompt states that the model may only use information explicitly present in the provided report, and must not infer, estimate, or add details from general medical knowledge.

b) Mandatory null / unknown handling

If a field cannot be found in the source text, the model must output null (or an empty list) instead of guessing a plausible value, and must log the missing field in `information_not_found`. This turns silence into a required, checkable output rather than letting the model quietly fill gaps.

c) Extractive, not generative, framing

The task is framed as extraction and light rephrasing, not clinical reasoning. The model is told it is not diagnosing, predicting outcomes, or adding recommendations beyond what the report already states.

d) Source-anchoring

Each finding and follow-up item includes a short excerpt copied from the original report. This lets a human reviewer verify grounding at a glance and lets the system automatically reject any item whose excerpt does not actually appear in the source text.

e) Constrained vocabulary

The model is instructed to reuse the report's own terms rather than translating them into a diagnosis or conclusion that is not explicitly stated (for example, not converting "elevated WBC" into "infection" unless the report itself says so).

f) Few-shot examples of correct "missing data" behaviour

The prompt includes one or two examples where a field is legitimately absent, showing the correct output of null plus an entry in `information_not_found`. This calibrates the model against always trying to fill every field.

g) Post-generation validation

- Schema validation - reject any output that does not match the fixed JSON structure.
- Excerpt verification - confirm each `source_excerpt` string actually exists in the original report text.
- Human review flag - any follow-up action without a matching reason drawn from the report is flagged for manual check.

h) Safety framing at the system level

The system prompt states plainly that the output will be used in a clinical setting, that fabricated information could cause patient harm, and that omission is always preferred over invention when the model is uncertain.

3. Summary

Technique	Purpose
"Extract, do not infer" instruction	Limits generative drift beyond the source text.
null / information_not_found fields	Makes gaps visible instead of silently filled.
Source excerpt per item	Enables traceability and automated verification.
Few-shot missing-data examples	Calibrates the model's handling of gaps.
Schema + excerpt validation	Catches residual fabrication programmatically.
Extraction framing over diagnosis	Prevents added clinical judgement not in the source.

Together, these constraints - structural design, explicit uncertainty handling, and verifiable grounding - are what actually prevent hallucination in a clinical summarization setting, rather than simply instructing the model to "be accurate."

4. Sample Implementation Source Code

```
from transformers import pipeline
import json
```

```
summarizer = pipeline("text2text-generation",
                      model="google/flan-t5-base")
```

```
report = "Patient has fever for 3 days. BP 120/80. No diabetes history."
prompt = "Extract only the information explicitly present in the report and return a valid JSON
summary. Use null if information is missing."
```

```
response = summarizer(prompt + report, max_new_tokens=200)
output = response[0]['generated_text']
parsed = json.loads(output)
print(json.dumps(parsed, indent=2))
```

5. Conclusion

The structured JSON design and prompt engineering approach ensures that the language model extracts only information present in the medical report. Using schema validation, null handling, and source-grounded extraction improves reliability and reduces hallucinations, making the solution suitable for healthcare documentation workflows.